Proyecto: Sport Analytics - Prevención de Sobrecarga y Optimización del Rendimiento Deportivo

1. Presentación y Alineación del Proyecto

Título provisional

AthletaGuard: Sistema Inteligente de Analítica Deportiva y Prevención de Sobrecarga

Elevator Pitch

AthletaGuard es un proyecto de IA y Big Data para diseñar, desarrollar y entrenar modelos de Machine Learning sobre series temporales deportivas (frecuencia cardíaca, variabilidad del pulso, volumen y RPE). El sistema procesa métricas de entrenamiento para predecir fatiga y riesgo de sobrecarga, generando recomendaciones adaptativas de intensidad.

El Tema y el Problema

- **Situación actual:** Atletas y entrenadores acumulan datos de GPS, pulsómetros y cargas de trabajo, pero la fatiga suele evaluarse mediante reglas estáticas o intuición.
- **Impacto:** La dificultad para predecir la acumulación de fatiga puede provocar sobreentrenamiento, lesiones y pérdida de progreso.
- **Valor:** Un modelo predictivo permite anticipar picos de fatiga, personalizar la carga semanal y optimizar el rendimiento.

Objetivo Principal del Proyecto

Construir un pipeline de Big Data y entrenar un modelo propio de Machine Learning/Deep Learning que procese series temporales de carga física para clasificar el nivel de riesgo de sobreentrenamiento y generar recomendaciones adaptativas.

2. Alcance Inicial (IN / OUT)

IN (Lo que SÍ incluye el proyecto)

- Adquisición y preprocesamiento de datasets de entrenamiento: frecuencia cardíaca, RPE, volumen y frecuencia de sesiones.
- Ingeniería de características: ventanas móviles, ratios de carga aguda/crónica (ACWR) y variaciones de HRV.
- Entrenamiento, ajuste y evaluación de modelos ML/DL (Random Forest, LightGBM, XGBoost, LSTM/1D-CNN) desde datos sin procesar, en local o Google Colab.
- Serialización e integración del modelo mediante API REST.
- Dashboard web para monitorizar carga e inferencias en tiempo real.

OUT (Lo que NO incluye el proyecto)

- APIs o modelos preentrenados de terceros ni cajas negras comerciales.
- Integración directa en producción con Garmin/Strava; se trabajará mediante datos brutos procesados en batch.
- Diagnósticos médicos o tratamientos fisioterapéuticos.

Despliegues cloud de pago con arquitecturas complejas.

3. Usuarios y Partes Interesadas (Stakeholders)

Stakeholder	Rol	Interés Principal	Riesgo
Usuarios finales	Deportistas / Atletas	Controlar su carga y evitar sobreentrenamiento.	Una mala interpretación puede generar predicciones erróneas.
Decisores	Entrenadores físicos	Planificar cargas con datos.	Una predicción incorrecta puede llevar a ajustar mal el entrenamiento.
Beneficiarios	Clubes deportivos	Mejorar rendimiento y reducir lesiones.	Los errores pueden afectar a la planificación deportiva.

4. Propuesta de Solución (Visión Técnica de Alto Nivel)

La solución integra 4 bloques funcionales centrados en el flujo de trabajo de Machine Learning e ingeniería de datos:

1. **Pipeline de Ingesta y Feature Engineering (Data Layer):** extracción, limpieza, imputación y generación de variables temporales.

2. **Entrenamiento y Ajuste del Modelo Propio (ML Engineering Layer):** Entrenamiento desde cero en Python/Jupyter/Colab, con validación Time Series Split y ajuste de hiperparámetros.
3. **Servicio de Inferencia Backend (API Layer):** API REST con FastAPI que cargará el modelo (.pkl/.onnx) y realizará predicciones sobre nuevos datos.
4. **Dashboard de Monitorización (Frontend Layer):** Interfaz web para visualizar métricas, evolución de fatiga y alertas tempranas.

5. Datos: Fuentes, Tipos y Riesgos Iniciales

- **Fuentes candidatas:** Datasets públicos de Kaggle (Running & Cycling Performance Logs, HRV & Training Load Time-Series) y datos de investigación deportiva de PhysioNet.
- **Tipo de datos:** Series temporales tabulares con fecha, duración, frecuencia cardíaca media/máxima, RPE, potencia y descanso.
- **Riesgos y aspectos legales/PII:** Se utilizarán datos anonimizados con licencias abiertas o académicas (MIT / Creative Commons). No se tratarán datos personales identificables que requieran RGPD avanzado en esta fase.

6. Viabilidad Técnica y Análisis de Riesgos

Análisis de Viabilidad

- **Partes sencillas:** Limpieza de datos, API de inferencia con FastAPI y dashboard web con React/Vue.
- **Partes difíciles:** Feature Engineering de series temporales, prevención de data leakage y ajuste de algoritmos para evitar overfitting en deportistas individuales.
- **Tecnologías:** Python (Pandas, NumPy, Scikit-Learn, XGBoost/LightGBM, PyTorch/TensorFlow opcional), FastAPI, PostgreSQL/SQLite, React/Vue/Angular y Git/GitHub.

Riesgos Principales y Plan B

1. **Riesgo 1: Bajo rendimiento del modelo inicial.** F1-Score/Accuracy insuficientes.
    - *Plan B:* Probar arquitecturas alternativas, como LSTM, y ajustar umbrales.
2. **Riesgo 2: Desequilibrio de clases en sobreentrenamiento.** Los días de sobrecarga son minoritarios.
    - *Plan B:* Aplicar técnicas de rebalanceo (SMOTE) y evaluar AUC-PR/Recall.
3. **Riesgo 3: Fuga de datos (Data Leakage).** Uso de datos futuros durante la validación.
    - *Plan B:* Utilizar Time Series Split respetando el orden cronológico.

7. Organización del Equipo y Reparto de Responsabilidades

El equipo asume un reparto de tareas centrado en el ciclo de vida completo del modelo desarrollado internamente:

- **David (Project Lead & ML Engineer / Data Scientist):**
  - Liderazgo y arquitectura ML.
  - Feature Engineering de series temporales (ACWR, medias móviles).
  - Entrenamiento, ajuste y evaluación de modelos.
- **Ivan (Backend & Data Pipeline Engineer):**
  - Pipeline de ingesta y limpieza.
  - Base de datos y API REST de inferencia (FastAPI).
  - Serialización y exportación de modelos.
- **Arnau (Frontend Specialist & BI / UI Analyst):**
  - Diseño e implementación del dashboard web interactivo.
  - Visualización de series temporales y gráficos de carga/fatiga.
  - Diseño UX de las alertas de sobrecarga.

8. Próximos Pasos Operativos (Próximos 7 Días)

1. Inicializar el repositorio Git con la estructura modular definida.
2. Seleccionar y descargar 2 datasets brutos candidatos de series temporales deportivas.
3. Crear un Jupyter Notebook inicial para EDA (Análisis Exploratorio) y validación de la calidad de los datos.
4. Definir el esquema del pipeline de extracción de características y primera prueba baseline.
5. Diseñar los primeros wireframes de la pantalla principal del dashboard.

Anexo: Acuerdo de Trabajo (Team Agreement)

- **Canales de comunicación:** Grupo de WhatsApp (avisos rápidos) y Discord/Teams (sesiones de trabajo).
- **Frecuencia de reuniones:** 2 reuniones semanales para control de hitos.
- **Reglas de repositorio Git:**
  - Prohibido hacer push directo a la rama main.
  - Integración mediante *Pull Requests* (PR) con revisión de al menos un compañero.
  - Documentación de experimentos de entrenamiento en el directorio docs/experiments.
  - Git: https://github.com/davidillo99/AthletaGuard